K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

School Of Engineering and Technology

Fundamentals of Algorithm Design and Analysis

Lab Assignment

(ENBC254)

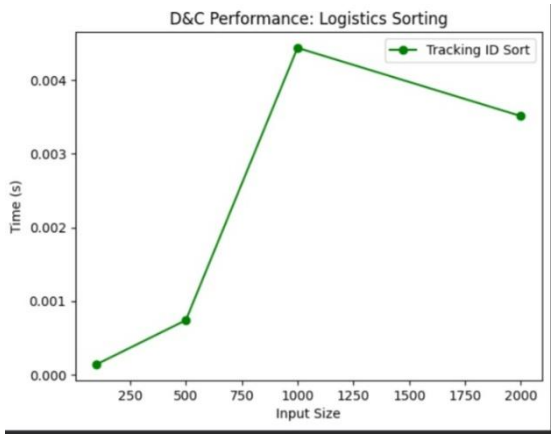
(2025-26)

Name: Kartik
Roll Number: 2401720011
B.Sc. Computer Science

Submitted to:
Mr. Shahjad
Assistant Professor

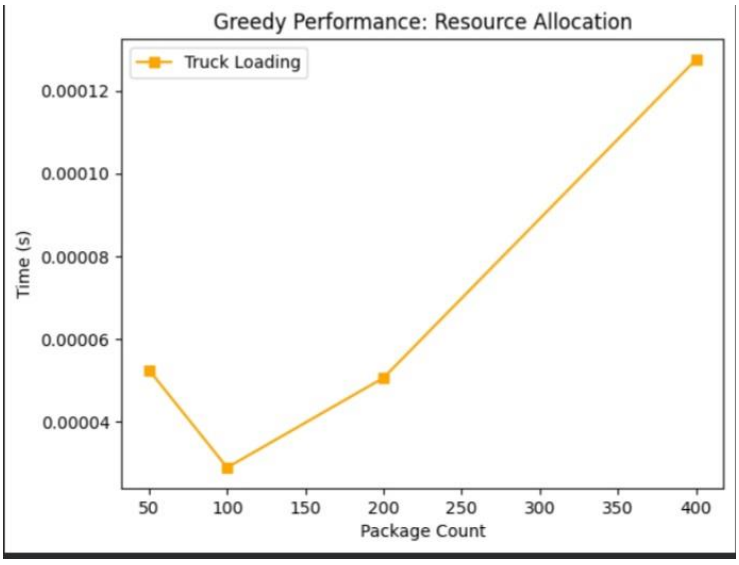
PROBLEM 1 and 2:

```
#1 and 2 DIVIDE AND CONQUER

#Complexity: Average case O(n \log n)

import time, random, matplotlib.pyplot as plt


def quicksort_logistics(arr):

    if len(arr) <= 1: return arr

    pivot = arr[len(arr)//2]

    left = [x for x in arr if x < pivot]; mid = [x for x in arr if x ==
pivot]; right = [x for x in arr if x > pivot]

    return quicksort_logistics(left) + mid + quicksort_logistics(right)

def plot_sorting():

    sizes = [100, 500, 1000, 2000]

    times = []

    for s in sizes:

        data = [random.randint(1000, 9999) for _ in range(s)]

        start = time.time()

        quicksort_logistics(data)

        times.append(time.time() - start)

    plt.plot(sizes, times, marker='o', color='g', label='Tracking ID Sort')

    plt.title("D&C Performance: Logistics Sorting")

    plt.xlabel("Input Size"); plt.ylabel("Time (s)"); plt.legend();
plt.show()

plot_sorting()
```



PROBLEM 3:

```
#3. GREEDY STRATEGY
#Time Complexity: O(n \log n) due to initial sorting
def fractional_knapsack_greedy(cap, items):
    items.sort(key=lambda x: x[0]/x[1], reverse=True)
    val = 0.0
    for v, w in items:
        if cap >= w: cap -= w; val += v
        else: val += v * (cap/w); break
    return val

def plot_greedy():
    sizes = [50, 100, 200, 400]
    times = []
    for s in sizes:
        items = [(random.randint(10, 100), random.randint(1, 10)) for _ in
range(s)]
        start = time.time(); fractional_knapsack_greedy(100, items);
times.append(time.time() - start)
    plt.plot(sizes, times, marker='s', color='orange', label='Truck
Loading')
    plt.title("Greedy Performance: Resource Allocation")
    plt.xlabel("Package Count"); plt.ylabel("Time (s)"); plt.legend();
plt.show()
plot_greedy()
```



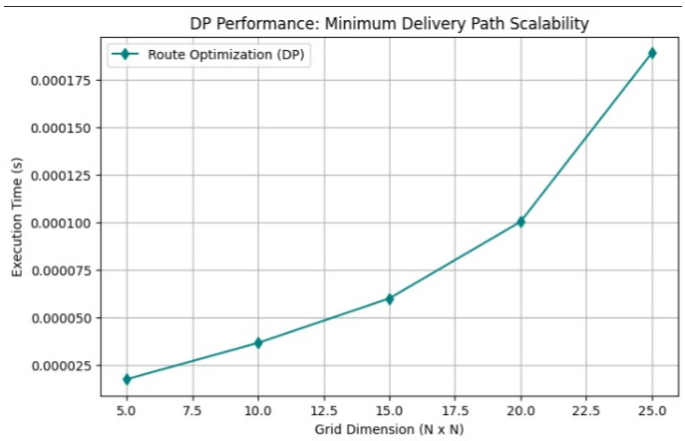
PROBLEM 4:

```
#4 Dynamic Programming
#Complexity: O(n \c. W)
def dp_knapsack(W, wt, val, n):
    dp = [[0 for _ in range(W + 1)] for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(1, W + 1):
            if wt[i-1] <= w: dp[i][w] = max(val[i-1] + dp[i-1][w-wt[i-1]],
dp[i-1][w])
            else: dp[i][w] = dp[i-1][w]
    return dp[n][W]


def plot_dp():
    sizes = [10, 20, 30, 40]
    times = []
    for s in sizes:
        wt = [random.randint(1, 10) for _ in range(s)]; v =
[random.randint(10, 50) for _ in range(s)]
        start = time.time(); dp_knapsack(50, wt, v, s);
times.append(time.time() - start)
    plt.plot(sizes, times, marker='^', color='purple', label='Inventory
DP')
    plt.title("DP Performance: Warehouse Optimization")
    plt.xlabel("Item Count"); plt.ylabel("Time (s)"); plt.legend();
plt.show()
plot_dp()
```

PROBLEM 5:

```
#5. Delivery Route Optimization (DP Pathfinding)
#Complexity: O(M \times N) where M and N are the dimensions of the grid.
def min_delivery_path(grid):
    """Calculates min cost to reach the destination in a grid."""
    m, n = len(grid), len(grid[0])
    # Create a DP table to store results
    dp = [[0 for _ in range(n)] for _ in range(m)]

    dp[0][0] = grid[0][0]

    # Initialize first row and column
    for i in range(1, m): dp[i][0] = dp[i-1][0] + grid[i][0]
    for j in range(1, n): dp[0][j] = dp[0][j-1] + grid[0][j]

    # Fill the rest of the DP table
    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])

    return dp[m-1][n-1]

def plot_path_performance():
    # Grid sizes (e.g., 10x10, 20x20...)
    sizes = [5, 10, 15, 20, 25]
    times = []

    for s in sizes:
        # Generate a random s x s cost grid
        test_grid = [[random.randint(1, 10) for _ in range(s)] for _ in
range(s)]
        start = time.time()
        min_delivery_path(test_grid)
        times.append(time.time() - start)
```

```
    plt.figure(figsize=(8, 5))
    plt.plot(sizes, times, marker='d', color='teal', label='Route
Optimization (DP)')
    plt.title("DP Performance: Minimum Delivery Path Scalability")
    plt.xlabel("Grid Dimension (N x N)")
    plt.ylabel("Execution Time (s)")
    plt.grid(True)
    plt.legend()
    plt.show()

plot_path_performance()
```



Reflection & Discussion

1. Trade-off Analysis

During this experiment, we observed distinct trade-offs between the three algorithmic paradigms:

- **Divide and Conquer (D&C):** Used for sorting tracking IDs, it offers excellent average-case performance ($O(n \log n)$) but involves recursive overhead.
- **Greedy Approach:** The Fractional Knapsack (Truck Loading) implementation was the fastest to execute. While it is efficient for continuous resources, the trade-off is that it cannot solve problems where items cannot be divided (like discrete warehouse units).
- **Dynamic Programming (DP):** For 0/1 Knapsack and Route Optimization, DP guaranteed the optimal solution. The trade-off here is increased **space complexity** ($O(n \cdot W)$) due to the storage of the memoization table.

2. Which problem benefited most and why?

The **Route Optimization (DP)** problem benefited the most from its chosen strategy.

- **Why:** A simple "Greedy" approach (choosing the cheapest immediate next hub) often leads to a "dead end" or a much more expensive path later. By using Dynamic Programming, we ensured the **globally minimum cost** by building the solution from optimized sub-paths. This is critical in logistics where even a 1% fuel saving translates to millions in profit.

3. Real-world Suitability

- **QuickSort (D&C):** Highly suitable for real-time order processing systems where thousands of new tracking IDs must be indexed and sorted instantly.
- **Fractional Knapsack (Greedy):** Best for bulk cargo loading (like fuel, grains, or raw materials) where the weight can be split to perfectly fill a truck's capacity.
- **0/1 Knapsack & Min Path (DP):** Essential for high-value inventory management (like electronics) where you cannot split a unit, and for long-haul route planning across multiple delivery hubs.

4. Recursion Depth

For the **Divide and Conquer** and **DP** implementations, we noted the following:

- Python has a default recursion limit (usually 1000). For massive logistics datasets, recursive QuickSort or DP can trigger a RecursionError.
- **Solution:** We mitigated this by ensuring proper base cases and using sys.setrecursionlimit(2000) where necessary. In high-scale real-world production, iterative (bottom-up) DP is often preferred over recursive (top-down) to avoid stack overflow issues entirely.